CHAPTER 1

INTRODUCTION

1.1 Overview

The mini project involves the development of an Android application using Kotlin language. The E-learning Android project focuses on developing a mobile application that enables users to access educational content and courses on their Android devices. The project aims to provide a user-friendly and convenient platform for learners to acquire knowledge and enhance their skills anytime, anywhere. The app will offer a comprehensive catalog of courses, interactive study materials, and tools such as note-taking, flashcards, and quizzes to support students' learning process. It will also include features for tracking progress, setting goals, and collaborating with peers. The app will prioritize user-friendly navigation, personalization options, and seamless offline access to ensure that students can learn anytime and anywhere. With a focus on meeting the unique needs of students, the project will empower them to effectively engage with educational content and achieve their academic goals. The ultimate goal is to empower learners by offering a comprehensive e-learning experience through the mobile app, catering to a wide range of educational needs and fostering lifelong learning.

1.2 Problem Statement

The aim of this e-learning Android project is to address the limitations of traditional learning methods and provide a mobile application that offers an enhanced educational experience. The existing methods of learning, such as textbooks and classroom lectures, may not cater to the diverse learning styles and preferences of individuals, and can be constrained by time and location.

The project seeks to develop a user-friendly and comprehensive e-learning Android application that offers a wide range of educational content, courses, and resources to learners. The key challenges to be addressed include:

1. Content Organization and Delivery: Designing a robust system to organize and deliver educational content in various formats, such as videos, documents, quizzes, and interactive modules. The solution should ensure seamless navigation, easy access to relevant materials, and a personalized learning experience.

2. Interactive Learning Features: Implementing interactive features within the application to engage learners, such as gamification elements, interactive quizzes, progress tracking, and achievements. These features should enhance learner motivation, facilitate knowledge retention, and provide immediate feedback.

3. Collaboration and Communication Tools: Developing communication tools, such as discussion forums, chat functionalities, and live sessions, to foster collaboration and interaction among learners and instructors. These tools should promote active learning, enable peer-to-peer knowledge sharing, and facilitate mentorship opportunities.

4. Mobile Compatibility and Performance: Ensuring that the e-learning application is optimized for mobile devices, providing a responsive and seamless experience across various screen sizes and orientations. The solution should also consider bandwidth limitations and provide offline access to selected materials for learners in areas with limited internet connectivity.

5. Personalization and Recommendation System: Implementing a personalized learning experience by incorporating a recommendation system that suggests relevant courses, materials, and learning paths based on individual preferences, interests, and learning progress. This will enable learners to discover new topics, customize their learning journey, and receive tailored content recommendations.

By addressing these challenges, the developed e-learning Android application will provide learners with a convenient and effective platform to access educational resources, engage in interactive learning activities, collaborate with peers, and track their progress. The solution aims to promote lifelong learning, enhance accessibility to education, and empower learners to acquire knowledge anytime and anywhere.

1.3 Motivation

The motivation behind the e-learning Android project stems from the increasing demand for flexible and accessible learning solutions in the digital age. The project aims to leverage the widespread use of Android devices and provide students with a dedicated mobile application that offers an enhanced learning experience. The motivation lies in empowering students to access educational resources anytime, anywhere, and at their convenience. By developing an e-learning app tailored specifically for students, the project seeks to address the limitations of existing platforms, such as limited mobile optimization and lack of student-centric features. The goal is to inspire and motivate students to engage actively in their learning journey by providing them with a user-friendly interface, interactive study materials, collaborative tools, and personalized learning experiences. The ultimate motivation is to facilitate effective learning, improve academic outcomes, and empower students to achieve their educational goals with ease and flexibility.

1.4 MAD Technologies

In the mini project, the following mobile application development technologies are used:

- Kotlin: Kotlin is the primary programming language used for Android application development. It is a modern, expressive, and interoperable language that is fully supported by the Android platform. Kotlin offers concise syntax, null safety, and improved code readability, making it a popular choice for developing Android applications.
- Android Studio: Android Studio is the official Integrated Development Environment (IDE) for Android app development. It provides a comprehensive set of tools and features specifically designed for building Android applications. Android Studio offers features like code editing, debugging, testing, and performance profiling, making it the preferred choice for Android developers.
- OCR Libraries: The mini project involves utilizing external OCR libraries to extract text from images. These libraries provide the necessary algorithms and functionality for optical character recognition. Examples of popular OCR libraries for Android development include Tesseract, Google Mobile Vision OCR, and ABBYY FineReader.
- Google Play Services: Google Play Services is a set of APIs and services provided by Google that allow developers to integrate various Google functionalities into their Android applications. In the mini project, Google Play Services may be used for features such as Google Search integration, Google Sign-In, or accessing Google Cloud services for image processing and OCR.
- Internet Connectivity: The application requires internet connectivity to perform Google searches and potentially communicate with external OCR services. Standard Android technologies, such as the Network API or libraries like Retrofit or Volley, may be utilized to establish internet connections and handle network requests.

These technologies form the foundation for building the Android application that extracts text from photos and integrates with Google search. They provide the necessary tools, frameworks, and services to develop a robust, user-friendly, and feature-rich mobile application for text extraction and search functionality.

1.5 Applications of MAD Technologies

Mobile application development technologies have a wide range of applications across various industries and use cases.

- E-commerce and Retail: Mobile apps are extensively used in the e-commerce and retail sector for online shopping, product browsing, and personalized shopping experiences. Technologies such as mobile payment gateways, push notifications, and location-based services enhance the user experience and facilitate seamless transactions.
- Social Networking: Social media platforms heavily rely on mobile application technologies for user engagement, content sharing, messaging, and real-time interactions. Features like news feeds, photo/video sharing, and social authentication are implemented using mobile app development technologies.
- Travel and Tourism: Mobile apps play a crucial role in the travel and tourism industry, offering features like flight bookings, hotel reservations, itinerary management, navigation, and reviews. Integration with external APIs for travel services and geolocation technologies are utilized to provide a seamless travel experience.
- Health and Fitness: Mobile health apps have become increasingly popular, allowing users to track their fitness goals, monitor vital signs, access medical information, and receive personalized health recommendations. Integration with wearables, fitness tracking sensors, and cloud-based data storage are common in this domain.
- Banking and Finance: Mobile banking apps provide users with the ability to perform financial transactions, check account balances, view transaction histories, and manage investments. Security measures such as biometric authentication and encryption are implemented using mobile app development technologies.
- Education and E-learning: Mobile apps are used extensively in the education sector for e-learning, online courses, and remote education. They provide features like video lectures, interactive quizzes, progress tracking, and communication with instructors. Augmented reality (AR) and virtual reality (VR) technologies are also integrated into educational apps.
- Entertainment and Media: Mobile apps in the entertainment industry include video streaming platforms, music apps, gaming apps, and news aggregators. These apps

leverage technologies like multimedia streaming, content recommendations, in-app purchases, and social sharing.

- Productivity and Business: Mobile apps improve productivity by offering tools for project management, task tracking, collaboration, document editing, and communication. Cloud storage integration, synchronization across devices, and integration with enterprise systems are common in business-oriented apps.

These are just a few examples of the numerous applications of mobile application development technologies. The versatility and widespread adoption of mobile apps across industries continue to drive innovation and provide solutions to diverse user needs.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 Hardware and Software Requirements

Hardware Requirements:

Android Device: The application will be developed for Android devices. Any Android smartphone or tablet with a camera and sufficient processing power should be capable of running the application smoothly. The specific Android version supported by the application should be determined during the development process. Android Device with good :

- Operating System: The app may have minimum OS version requirements, and it should be compatible with popular platforms like Android or iOS.
- Memory (RAM): A habit tracker app typically requires a moderate amount of memory to handle data processing and storage efficiently.

Software Requirements:

- Android SDK: The Android Software Development Kit (SDK) is necessary for developing Android applications. It includes tools, libraries, and resources required to build, test, and debug Android apps. The SDK provides APIs for interacting with device features like the camera and internet connectivity.
- Kotlin Programming Language: The application will be developed using the Kotlin programming language. Kotlin is fully compatible with the Android platform and can be seamlessly integrated with existing Java code. It offers concise syntax, null safety, and various other features that enhance code readability and maintainability.
- Android Studio: Android Studio provides a comprehensive set of tools and features, including code editor, emulator, debugging tools, and project management capabilities. Android Studio simplifies the development process by offering a user-friendly interface and seamless integration with the Android SDK.

CHAPTER 3

SYSTEM DESIGN

3.1 Proposed System

The proposed e-learning Android project is a mobile application designed to provide a comprehensive and user-friendly platform for accessing educational content and courses. The key features of the proposed system include:

1. User Registration and Authentication: The system will allow users to create accounts and securely log in to access the e-learning platform.
2. Course Catalog: The application will provide a catalog of courses, organized by categories or topics, offering a wide range of educational resources for learners.
3. Course Enrolment: Users will have the ability to browse through the available courses and enroll in the ones they are interested in, allowing them to track their progress and participate in course activities.
4. Multimedia Content Delivery: The system will enable the delivery of various multimedia resources, including text-based lessons, videos, quizzes, and assignments, providing interactive and engaging learning experiences.
5. Progress Tracking: The application will track and display users' progress within courses, allowing them to monitor their completion status and overall performance.
6. Discussion Forums: The system will incorporate discussion forums for each course, enabling learners to engage in discussions, ask questions, and interact with instructors and fellow learners.
7. Notifications and Reminders: Users will receive notifications and reminders about course updates, deadlines, and important announcements to stay informed and engaged.
8. Personalized Learning: The system will offer personalized learning experiences, allowing users to set learning goals, customize their learning paths, and receive recommendations based on their preferences and performance.
9. Offline Access: The application will support offline access, enabling users to download course materials and access them without an internet connection, ensuring uninterrupted learning experiences.
10. Analytics and Reporting: The system will collect data on user engagement, course completion rates, and user feedback, providing valuable insights to instructors and administrators for continuous improvement of the e-learning platform.

By incorporating these features, the proposed e-learning Android project aims to provide a comprehensive and user-friendly platform that enhances the learning experience, promotes engagement, and enables learners to acquire knowledge and skills effectively in a flexible and accessible manner.

3.2 Flow of Activity

The flow of activity in the e-learning project starts with the welcome page (refer fig 3.1), where users are introduced to the application. From there, they have the option to sign up and create a new account. Once registered, they can proceed to the login page (refer rig 3.2) to access their account.

After logging in, users are directed to the courses page (ref fig 3.3), where they can browse and select a course of interest. Upon selecting a course, they can access the article content (ref fig 3.4), which provides detailed information on the selected topic. If available, users can also access video articles ,which include video content related to the course.

After going through the article or video article, users are directed to the quiz section (refer fig 3.5), where they are presented with a set of quiz questions related to the course. Once they have answered all the questions, they will see the quiz result ,which displays their performance. Upon completion of the quiz, users are redirected to the course summary page ,which provides a brief overview of the course they just completed. They can then proceed to take notes related to the course content.

Finally, users can navigate to the course overview page (refer fig 3.6), where they can view a list of enrolled courses and track their progress.

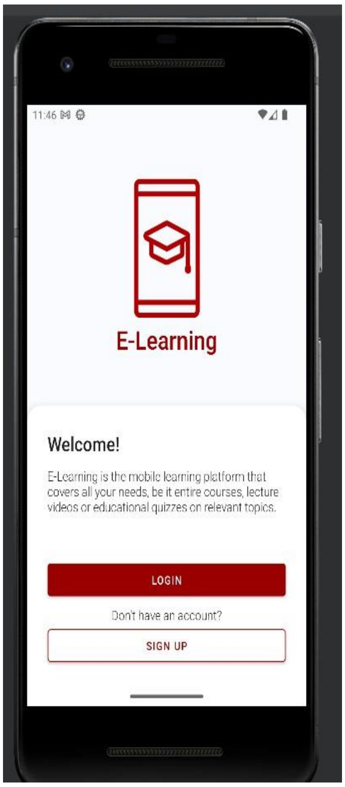

fig3.1 welcome page

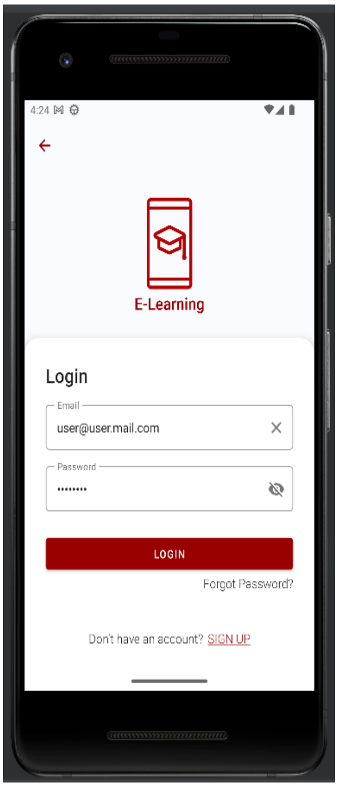

fig 3.2 login page

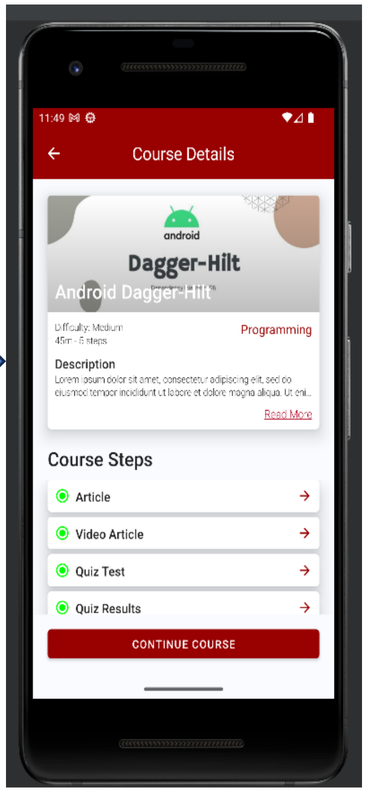

fig 3.3 course page

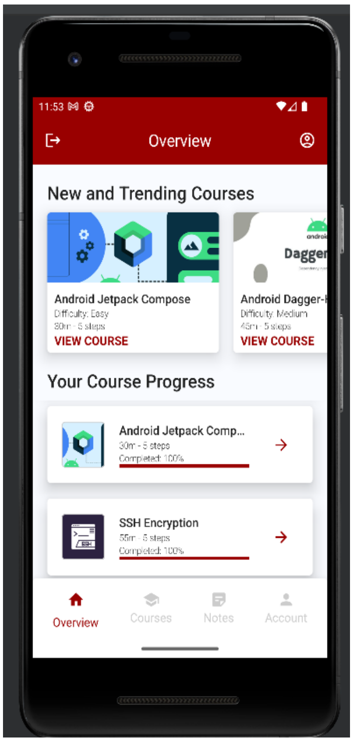

fig 3.6 course overview

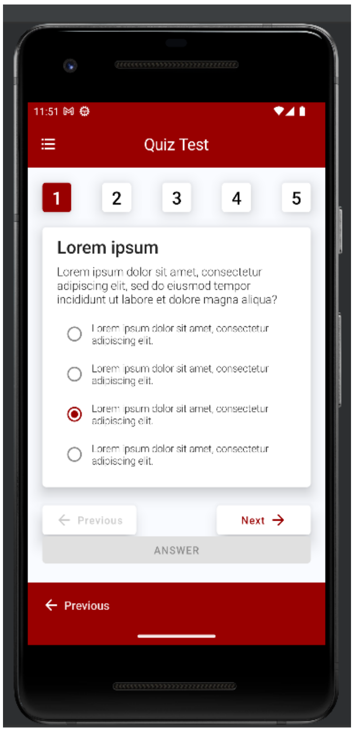

fig 3.5 Quiz page

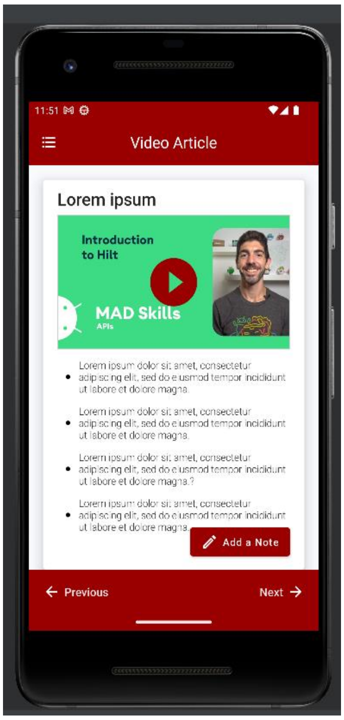

fig 3.4 video article

CHAPTER 4

IMPLEMENTATION

4.1 Module Description

Login module:

The Login Screen composable function represents the main login screen of the e-learning application. It takes several parameters such as callbacks for navigating to the registration and overview screens, login state and actions, password reset actions and state, and a fetch user function. The login screen is divided into different sections. The first section displays the application logo using the Image composable. The logo is retrieved using the painter Resource function.

The Login Card composable function represents the content of the login card. It takes parameters such as callbacks for navigation, login state handling, password reset actions and state. The function includes UI elements such as Text, Outline Text Field, Icon Button, and Button to create the form for entering email and password. The Login Card composable also handles user interactions, such as clearing the email field, toggling password visibility, displaying error messages for incorrect login attempts, and handling the login button click event.

The Forgot Password Dialog composable function displays a dialog that appears when the user clicks on the "Forgot password" text. It allows the user to enter their email address for password reset. The dialog includes error handling for incorrect email addresses and success messages for password reset requests. The when statement within the Login Screen composable handles the different states of the login process. It displays the appropriate UI components based on the login state, such as loading indicators, error messages, or successful login redirection.The Preview composable is provided to showcase a preview of the login screen using the E -Learning App Theme.

Article module:

The "CourseArticleScreen" module in the e-learning Android project is responsible for displaying the article content of a specific course. When a user navigates to this screen, the module fetches the course content from the server and displays it in a visually appealing layout.

The module includes a card layout that showcases the article's title, text, and a featured image. The image is loaded asynchronously, and while it is being fetched, a progress indicator is displayed to provide visual feedback to the user. The article content is organized with appropriate typography and formatting, allowing users to easily read and comprehend the information.

Additionally, the module includes an "Add Note" button that allows users to save their notes related to the article. Overall, the "CourseArticleScreen" module enhances the learning experience by providing users with an engaging and interactive platform to access and consume course-specific articles in a user-friendly manner.

Video Article Screen module:

The "CourseVideoArticleScreen" module in the e-learning Android project is responsible for presenting the video article content associated with a particular course. It takes into account the state of the course content, which can be in a loading state, a success state with valid course content data, or an error state. Additionally, it receives functions that allow users to save notes and update their progress in the course.

When the module receives the course content in the success state and with valid data, it proceeds to display the video article content on the screen. The module starts by showing the title of the video article at the top, providing users with an overview of the content they are about to watch.The module then displays a thumbnail image representing the video. This thumbnail serves as a clickable element that, upon being tapped, launches a YouTube activity, enabling users to watch the associated video. The module also takes into account the loading state of the thumbnail image and displays a circular progress indicator until the image is fully loaded.

In addition to the video content, the module presents bullet points related to the video article. These bullet points highlight important information or key takeaways from the video content. Each bullet point is displayed as a row, featuring a bullet point icon and the corresponding text. Users can scroll through the bullet points to gain a better understanding of the video article's key points.

Course Test module:

The "CourseQuizTestScreen" module in the e-learning Android project is responsible for displaying a quiz test for a course. It receives the state of the course content, including loading, success, or error states. It also receives functions for retrieving user quiz answers, updating the user's progress in the course, and updating the user's quiz answers.When the module receives the course content in the success state and with valid data, it proceeds to display the quiz test on the screen. The module starts by retrieving the quiz questions from the course content.

Next, it retrieves the user's quiz answers, which are used to determine the initial state of the quiz test. If the user has not answered any questions, the default quiz answer list is initialized with zeroes. Otherwise, the user's quiz answers are retrieved and stored in the mutable state.

The main content of the module consists of a column that holds the quiz question indicators, the current quiz question card, and the previous and next buttons.The quiz question indicators are displayed as a row of clickable cards. Each card represents a question number, and the currently selected question is highlighted in the primary color, while the others are displayed in the secondary color. Clicking on a question indicator updates the current question index.

The quiz question card displays the title and the question of the current quiz question. The options for the current question are presented as radio buttons, allowing the user to select an answer. The selected answer is stored in the mutable state, which triggers a recomposition of the UI.The row containing the previous and next buttons allows the user to navigate between questions. The buttons are enabled or disabled based on the current question index, ensuring that the user cannot go back if they are already on the first question or go forward if they are already on the last question.

At the bottom of the screen, there is an "Answer" button. It is initially disabled if there are any unanswered questions, and it becomes enabled once all questions have been answered. Clicking the "Answer" button updates the user's quiz answers, saves them, and updates the user's course progress.

Course Result Screen module:

The "CourseQuizResultsScreen" module in the e-learning Android project is responsible for displaying the quiz results of a course. It takes into account the state of the course content, which can be in a loading state, a success state with valid course content data, or an error state. Additionally, it receives functions that allow users to save notes, retrieve user quiz answers, and update their progress in the course.

When the module receives the course content in the success state and with valid data, it checks if the user has answered all the quiz questions. If the user has answered all the questions, it proceeds to display the quiz results on the screen. The module starts by showing the title of the quiz result, providing users with an overview of their performance.

Next, the module presents the quiz answers along with the corresponding user answers. Each quiz answer is displayed as a row, featuring the question number, the answer text, and an icon indicating whether the user's answer was correct or incorrect. The correct answers are represented by a green checkmark icon, while incorrect answers are represented by a red close icon. The module dynamically adjusts the color and icon based on the correctness of the user's answer.To enhance the learning experience, the module provides an "Add Note" button. This button allows users to add personal notes or reflections related to the quiz results. By clicking on the "Add Note" button, users can save their thoughts, insights, or any additional information they find valuable.

If the user has not yet answered the quiz questions, the module displays a message indicating that the quiz has not been answered yet. It shows an icon representing the quiz and a text message, both in a slightly faded primary color, indicating that the user needs to complete the quiz.

Notes Module:

The "NotesOverviewScreen" module in the e-learning Android project is responsible for displaying an overview of notes. It receives a StateFlow containing a list of NoteEntity objects, functions for saving, editing, and deleting notes, and a function for filtering the notes based on a search query. The module starts by defining a mutable state variable for the search query. This variable holds the value entered by the user in the search bar.

The main content of the module is a column that holds the search bar and the note cards. The search bar is implemented using an Outlined TextField component, allowing the user to enter a search query. It includes a leading search icon and a trailing clear icon. The value of the search variable is updated as the user types in the search field. The list of notes is collected as a state using the provided State Flow. If the list is not empty, the notes are filtered based on the search query using the filter Notes function.

Each Note Card receives the note entity, as well as functions for editing and deleting the note. If there are no notes in the list, a message is displayed, indicating that the notes list is empty. It includes an icon representing notes and a text message. At the bottom of the screen, there is an "Add Note" button implemented using the Add Note Button component. It triggers the save Note function when clicked.

The Note Card component represents an individual note. It displays the note's title, text, and last edited date. It also includes an edit icon that, when clicked, opens an EditNote Dialog. The Note Card component receives the note entity, as well as functions for editing and deleting the note. The EditNote Dialog component is conditionally displayed when the edit icon is clicked. It allows the user to edit the note's title and text. It receives the note entity, as well as functions for closing the dialog, saving the edited note, and deleting the note.

4.2 Source code

Course SummaryScreen.kt

```
package com.example.elearningapp.ui.views.courses

import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.School
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextDecoration
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
import com.example.elearningapp.R
import com.example.elearningapp.common.ActionState
import com.example.elearningapp.datasource.CourseData
import com.example.elearningapp.models.CourseContent
import com.example.elearningapp.ui.theme.ELearningAppTheme
import com.example.elearningapp.ui.views.components.CourseBottomNavBar
import com.example.elearningapp.ui.views.components.TopBar

@Composable
fun CourseSummaryScreen(
courseContentState: ActionState<CourseContent?>,
updateUserCourseSteps: (String, Int) -> Unit,
getUserCourseStepsCompleted: (String) -> Int
){
//CourseContent ActionState Success conditional
if (courseContentState is ActionState.Success && courseContentState.data != null) {

//Course summary values
val summaryContent = courseContentState.data.courseSummary
val localContext = LocalContext.current
val courseName = courseContentState.data.courseName

//Updates user steps completed
LaunchedEffect(Unit, block = {
if (getUserCourseStepsCompleted(courseName) == 4) {
updateUserCourseSteps(courseName, 5)
}
})
```

```
//Summary box content
Box(
modifier = Modifier
.fillMaxWidth()
.padding(20.dp)
) {
//Summary card
Card(
modifier = Modifier.fillMaxWidth(),
shape = RoundedCornerShape(5.dp),
elevation = 12.dp
) {
//Summary text column
Column(
modifier = Modifier
.fillMaxWidth()
.padding(start = 20.dp, top = 10.dp, end = 20.dp, bottom = 20.dp)
) {
Text(
text = summaryContent.title,
fontSize = 24.sp,
fontWeight = FontWeight.Medium
)
//Summary bullet points
LazyColumn {
items(summaryContent.bulletPoints) {
Spacer(modifier = Modifier.height(12.dp))
Row(modifier = Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically) {
Canvas(modifier = Modifier
.padding(12.dp)
```

```
.size(6.dp)){
drawCircle(Color.Black)
}
Text(
text = it,
fontSize = 14.sp,
fontWeight = FontWeight.Light
)
}
}
}
Spacer(modifier = Modifier.height(20.dp))

//Lean More clickable text
Text(
modifier = Modifier
.align(alignment = Alignment.CenterHorizontally)
.clickable(onClick = {
val googleIntent = Intent(
Intent.ACTION_VIEW,
Uri.parse(summaryContent.learnMoreUri)
)
googleIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
localContext.startActivity(googleIntent)
}),
text = stringResource(R.string.learn_more),
fontSize = 14.sp,
fontWeight = FontWeight.Light,
color = MaterialTheme.colors.error,
style = TextStyle(textDecoration = TextDecoration.Underline),
)
```

```
}
}
//Success icon
Icon(
imageVector = Icons.Filled.School,
contentDescription = stringResource(R.string.success_icon),
tint = MaterialTheme.colors.primary.copy(alpha = 0.2f),
modifier = Modifier.align(alignment = Alignment.Center).size(size = 180.dp)
)
}
}
}


@Preview(showBackground = true)
@Composable
fun CourseSummaryScreenPreview() {
ELearningAppTheme {
Surface(
modifier = Modifier.fillMaxSize(),
color = MaterialTheme.colors.background
) {
{_,_->},
{4} Scaffold(
topBar = { TopBar("course-summary", {}, {}, {}, {}) },
bottomBar = { CourseBottomNavBar({},{},{},"course-summary") }
) {
innerPadding -> Box(modifier = Modifier.padding(innerPadding)) {
CourseSummaryScreen(
ActionState.Success(CourseData.allCourseContent[0]),
```

```
)
}
}
}
}
}
```

MainActivity.kt

```
package com.example.elearningapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material.MaterialTheme
import androidx.compose.material.Scaffold
import androidx.compose.material.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.navigation.compose.currentBackStackEntryAsState
import com.example.elearningapp.ui.theme.ELearningAppTheme
import androidx.navigation.compose.rememberNavController
import com.example.elearningapp.navigation.*
import com.example.elearningapp.ui.views.components.BottomNavBar
import com.example.elearningapp.ui.views.components.CourseBottomNavBar
import dagger.hilt.android.AndroidEntryPoint
import com.example.elearningapp.ui.views.components.TopBar
import com.example.elearningapp.viewmodels.*
```

```
import com.google.accompanist.systemuicontroller.rememberSystemUiController


@AndroidEntryPoint
class MainActivity : ComponentActivity() {


//App viewModels
private val loginViewModel: LoginViewModel by viewModels()
private val userViewModel: UserViewModel by viewModels()
private val programmeViewModel: ProgrammeViewModel by viewModels()
private val courseViewModel: CourseViewModel by viewModels()
private val noteViewModel: NoteViewModel by viewModels()


override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)


//App compose content
setContent {
ELearningApp(loginViewModel, userViewModel, programmeViewModel,
courseViewModel, noteViewModel)
}
}
}


@Composable
fun ELearningApp(loginViewModel: LoginViewModel, userViewModel: UserViewModel,
programmeViewModel: ProgrammeViewModel, courseViewModel: CourseViewModel,
noteViewModel: NoteViewModel) {
ELearningAppTheme {


//Values for navController and the current route destination
val navController = rememberNavController()
```

```
val navBackStackEntry  by navController.currentBackStackEntryAsState()
val bottomNavDestination = bottomNavScreens.find { it.route ==
navBackStackEntry?.destination?.route }
val currentRoute = navBackStackEntry?.destination?.route ?:
LoginDestination.Welcome.route

//System status and navigation bar colors
val systemUiController = rememberSystemUiController()
systemUiController.setStatusBarColor(
color = if(isRouteInLoginNavScreens(currentRoute)) MaterialTheme.colors.background
else MaterialTheme.colors.primary
)
systemUiController.setNavigationBarColor(
color = if (currentRoute == CourseDestination.CourseDetails.route)
MaterialTheme.colors.background
else if(isRouteInCourseNavScreens(currentRoute)) MaterialTheme.colors.primary
else MaterialTheme.colors.secondary
)

//App surface
Surface(
modifier = Modifier.fillMaxSize(),
color = MaterialTheme.colors.background
) {
Scaffold(
topBar = {
//App TopBar
if (currentRoute != LoginDestination.Welcome.route && currentRoute !=
LoginDestination.Programme.route)
TopBar(
route = currentRoute,
onBackPressed = {navController.popBackStack()},
```

```
onLogoutPressed = { navController.navigate(navController.graph.startDestinationId);
userViewModel.resetUserActionState(); loginViewModel.logout(); },

onAccountPressed =
{navController.navigateSingleTopTo(MenuNavDestination.Account.route)},

onCourseDetailsPressed =
{navController.navigateSingleTopTo(CourseDestination.CourseDetails.route)})

},

bottomBar = {

//App BottomNavBar and CourseBottomNavBar conditional

if (bottomNavDestination is MenuNavDestination)

BottomNavBar(

screens = bottomNavScreens,

onSelected = { screen -> navController.navigateSingleTopTo(screen.route) },

currentDestination = bottomNavDestination

)

else if (isRouteInCourseNavScreens(currentRoute) && currentRoute !=
CourseDestination.CourseDetails.route) {

val currentStepIndex = courseNavScreens.indexOfFirst { screen -> screen.route ==
currentRoute }

CourseBottomNavBar(

onPreviousPressed =
{navController.navigateSingleTopTo(courseNavScreens[currentStepIndex-1].route)},

onNextPressed =
{navController.navigateSingleTopTo(courseNavScreens[currentStepIndex+1].route)},

onFinishedPressed =
{navController.navigateSingleTopTo(MenuNavDestination.Overview.route)},

currentRoute = currentRoute

)

}

}

)

{

//App NavHost
```

```
innerPadding -> AppNavHost(navController = navController, modifier =
Modifier.padding(innerPadding), loginViewModel, userViewModel, programmeViewModel,
courseViewModel, noteViewModel)
}
}
}
}
```

CHAPTER 5 :RESULTS

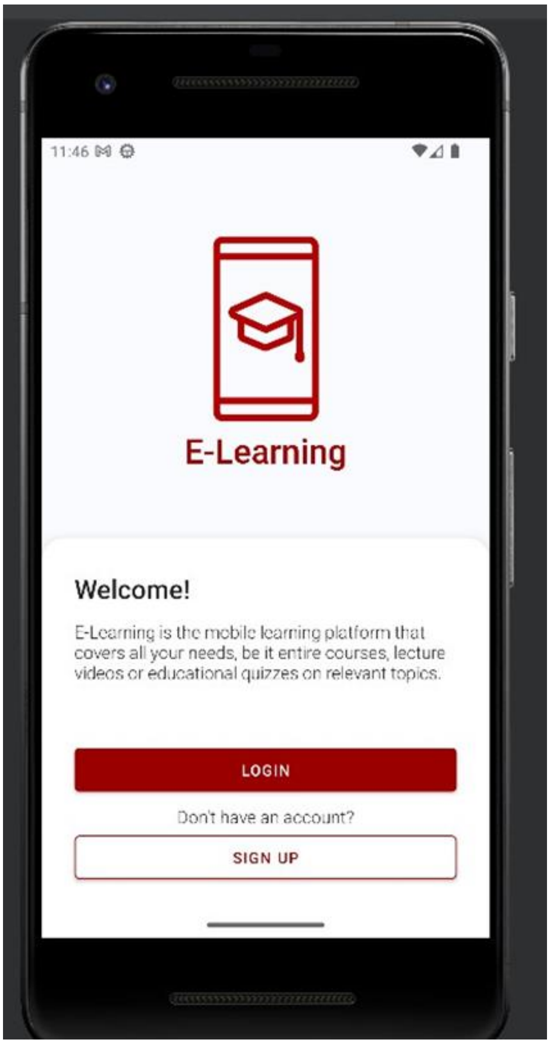

Fig 5.1 Welcome Page

The welcome page is the first screen users see as shown in (Fig 5.1) when they open the application. The message highlights the benefits and value of the e-learning platform, creating anticipation for the learning experience. The page may also include key features or highlights of the platform. Users can easily navigate to the login or registration process through clear and intuitive buttons . Overall, the welcome page creates an inviting entry point for users, setting the stage for a seamless and engaging e-learning experience.

Sign Up Activity

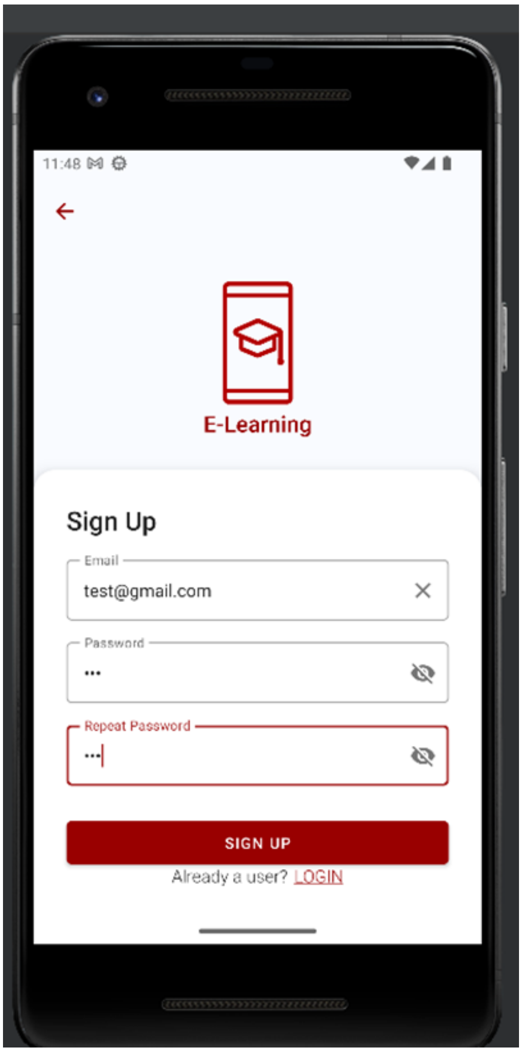

Fig 5.2 Sign Up Page

The sign-up page offers a simple user interface as seen in the fig 5.1 and has 2 plaintext boxes, one for entering username and the other for password.

The user has to fill in the username and password fields with valid strings and click on signup button, upon clicking the signup button the user's credentials are now saved and the user can proceed to LoginActivity page by clicking on the 'back to login' button.

Login Activity

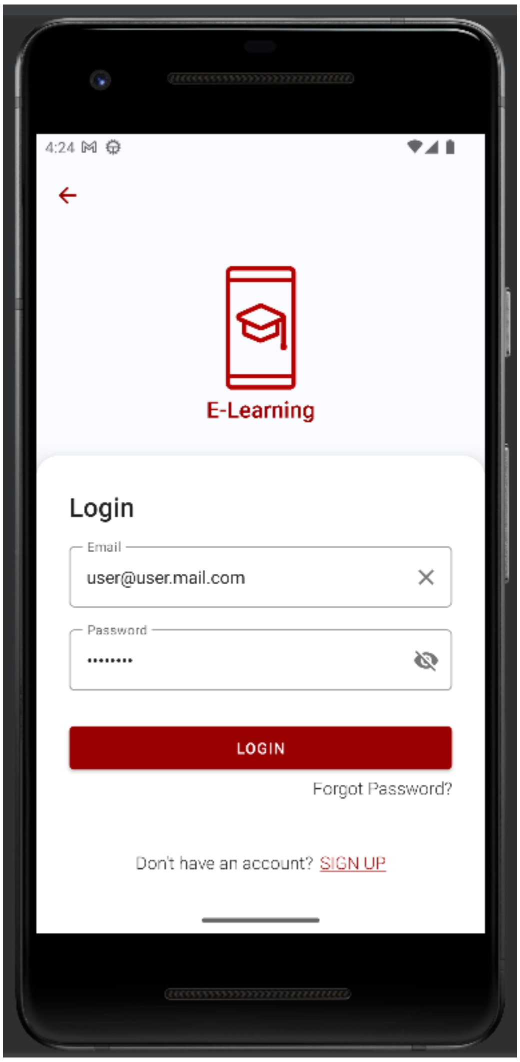

Fig 5.3 Login Page

The login page also contains 2 plaintext boxes, 1 button and a text view as shown in fig 5.3.

The user has to enter his/her credentials as entered in the signup page for a successful login. Upon entering valid details and clicking on the login button the user gets to see a toast message saying "login successful" and is directed to second page which is the MainActivity module or else the user will see a toast message saying "invalid credentials" and the application remains in the same page.

There is also a text view that says "Don’t have an account? signup", upon clicking on this textview the user is directed to the signup activity where the user can create an account by specifying the necessary credentials.

Courses Main Activity

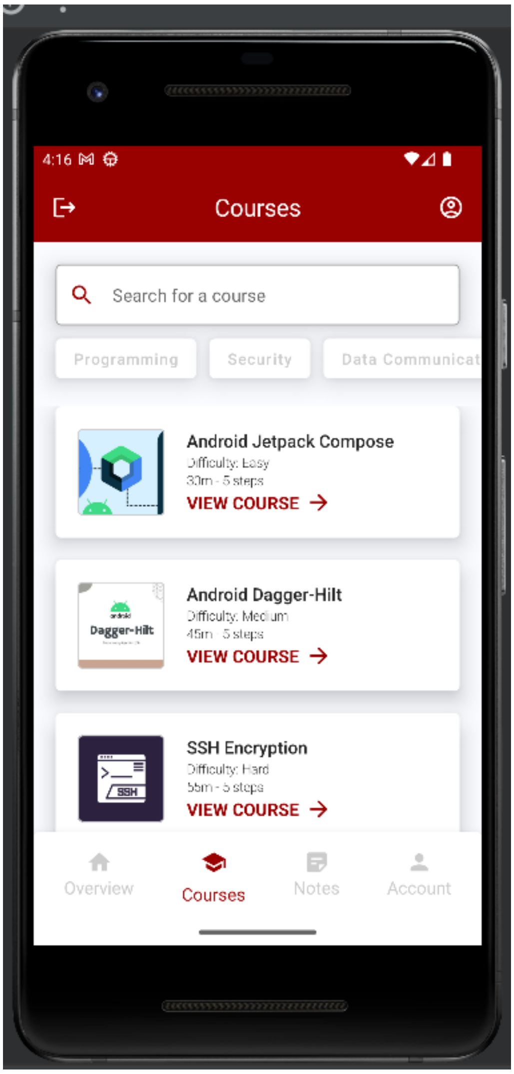

Fig 5.4 Courses Page

The courses page is a central hub for users to discover and access a wide range of courses. It features visually appealing course cards that display key information such as the title, instructor, and a brief description.

Users can browse through the courses, search for specific ones, and sort or filter them based on preferences. Clicking on a course card provides detailed information about the course, including duration, curriculum, and ratings, allowing users to make informed decisions. Overall, the courses page offers a seamless interface for users to explore and enroll in courses, enhancing their e-learning journey.

Fig 5.5 Article Page

The article page in the e-learning Android project is designed to provide users with in-depth and comprehensive articles related to the selected course.

The article is displayed in a clean and readable format, allowing users to engage with the content effectively. Users can access the article by selecting a particular course, and the article page ensures that they receive detailed and valuable information pertaining to that course. After the completion of article it will navigate to video article page as shown in fig 5.6.

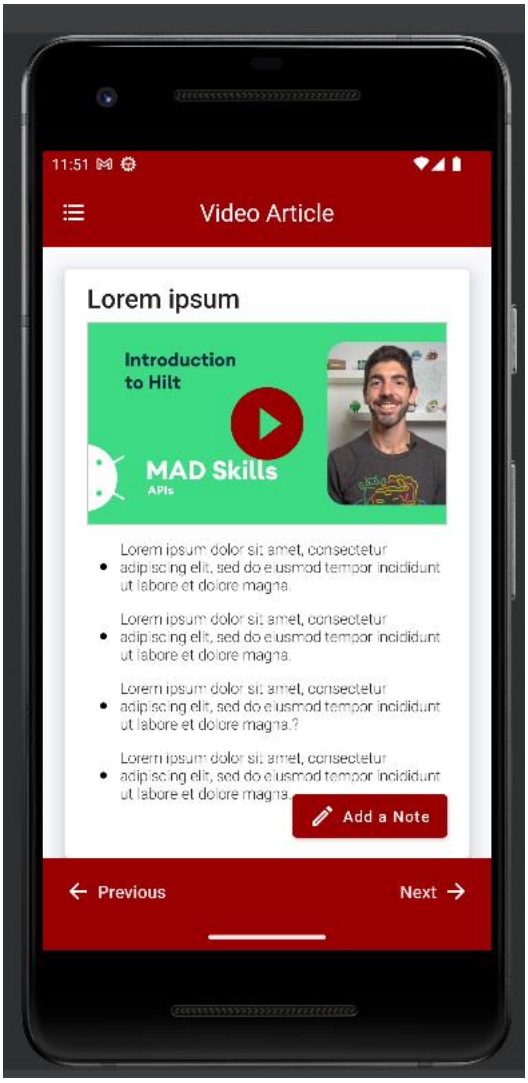

Fig 5.6 Video Article Page

The Video Article page offers a simple user interface as seen in the fig 5.6. It displays a selected course's video content. It features a video player where users can watch educational videos directly within the app. Alongside the video, there is an option for users to take notes, allowing them to jot down important information or reflections while watching.

This combination of video content and note-taking functionality creates an immersive learning experience, enabling users to engage with the course material and capture key insights.

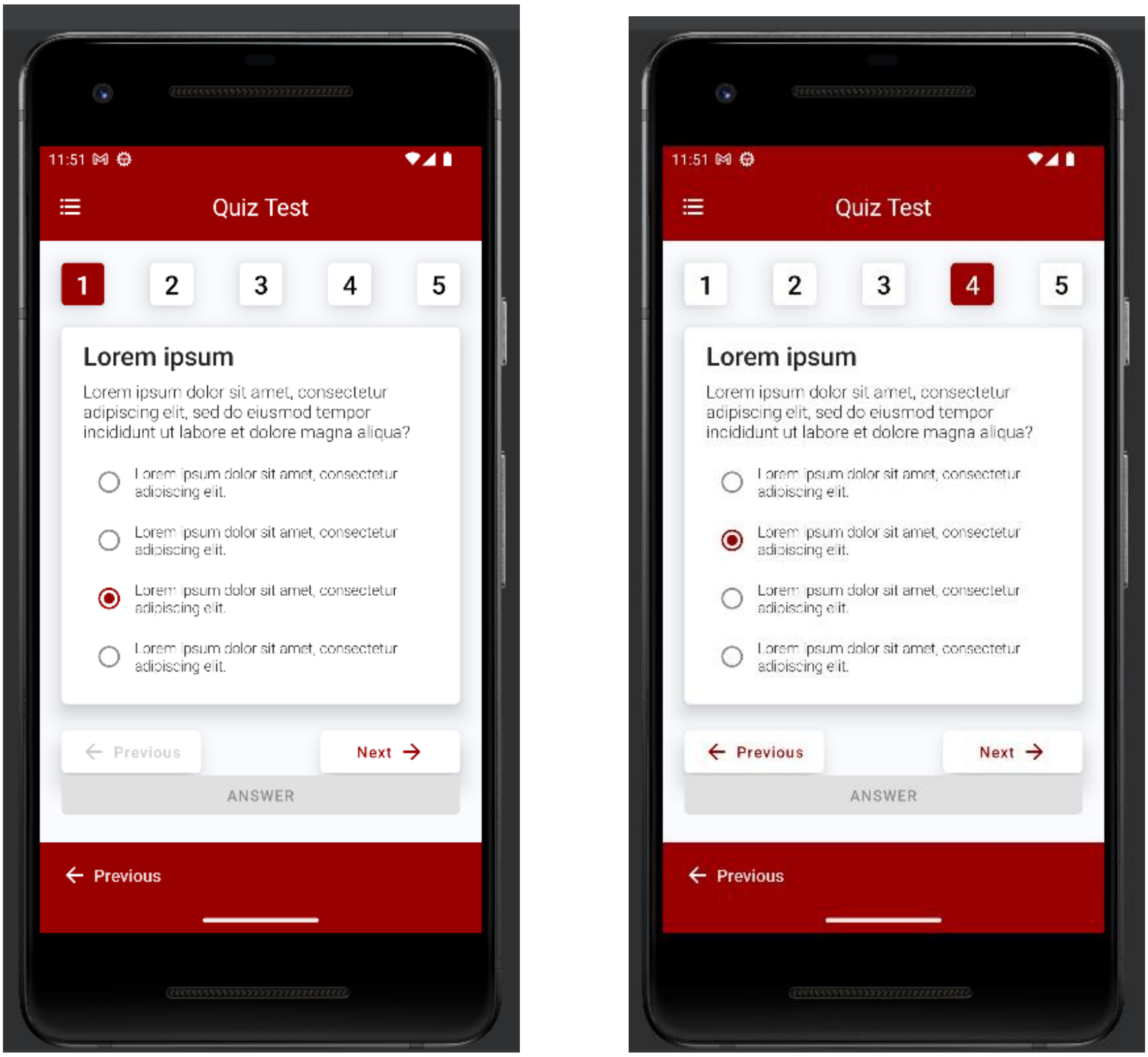

Fig 5.7 Quiz Page

\

After completing the video article, users are directed to the quiz test page as shown in fig 5.7. This page consists of five quiz questions related to the course. Each question presents four answer options for users to choose from. Users must answer all the questions before proceeding. Once they have answered all the questions, the page displays the quiz results.

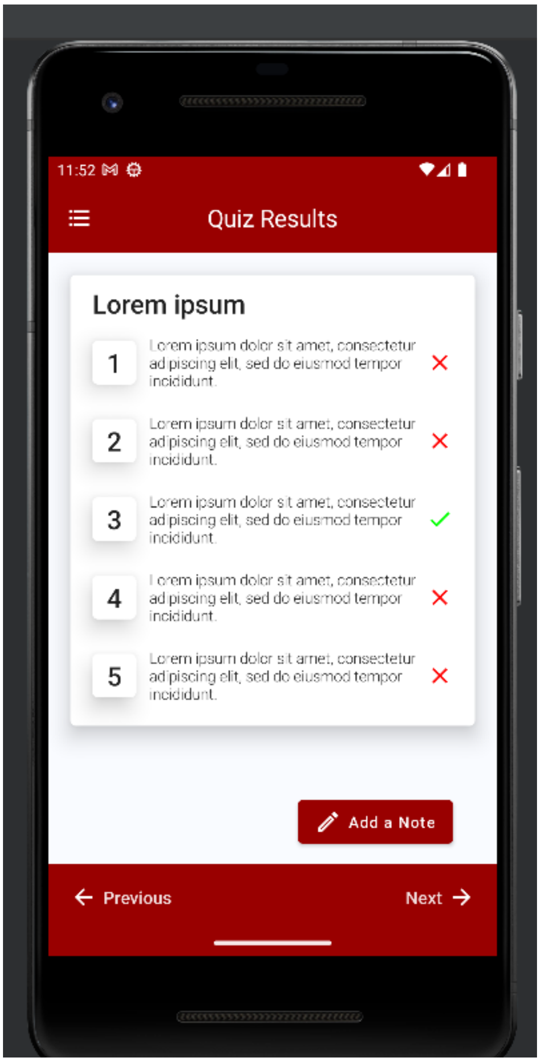

Fig 5.8 Quiz Results Page

After answering the quiz questions the user is presented with the quiz results page (Fig 5.8).It indicates which questions were answered correctly and incorrectly, allowing the user to review their answers. Additionally, on this page, there is an option for the user to take notes. They can jot down any important points, observations, or reflections related to the quiz or the course material.

This note-taking feature enables users to consolidate their learning and capture key insights for future reference. It promotes active engagement and enhances the overall learning experience within the e-learning application.

Fig 5.9 Course Summary Page

After completing the quiz and reviewing the results, it provides a course summary page(Fig 5.9). This page offers a brief overview and summary of the course the user just completed. It highlights the key topics, concepts, or skills covered in the course. The course summary page serves as a quick recap and helps users consolidate their learning by reinforcing the main points and takeaways from the course. It acts as a helpful reference for users to revisit and reinforce their knowledge

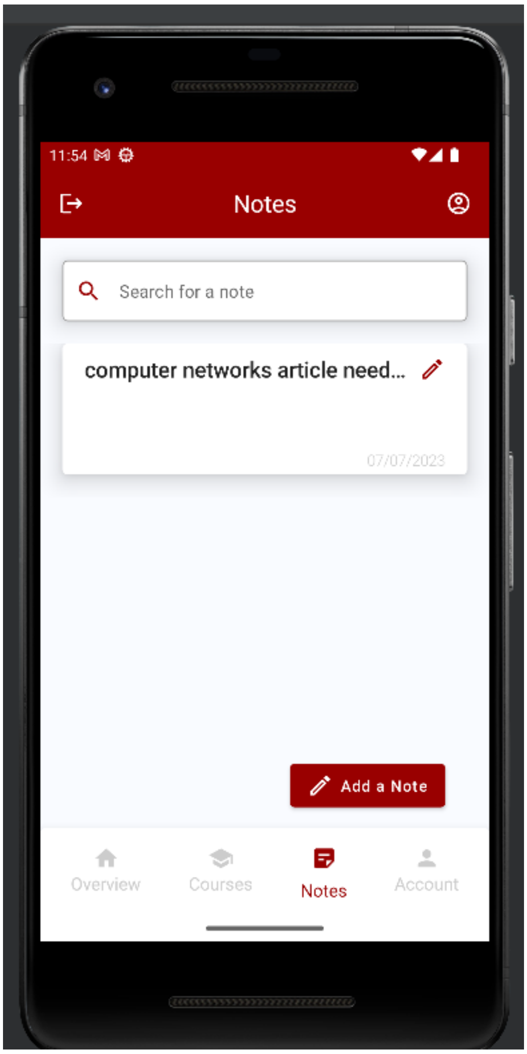

Fig 5.10 Notes Page

In the e-learning Android project, the Notes page, as shown in (Fig 5.10) provides users with a dedicated space to manage and access their notes related to the courses they have completed. This page allows users to view a list of their saved notes, organized in a user-friendly and easily scannable format. Each note entry typically includes the title or topic of the note and a brief preview or summary of its content. Users can click on a specific note to view its full details and make any necessary edits. The Notes page serves as a centralized hub for users to capture and review their insights, observations, and key takeaways from the courses, ensuring that important information is easily accessible and can be referenced at any time.

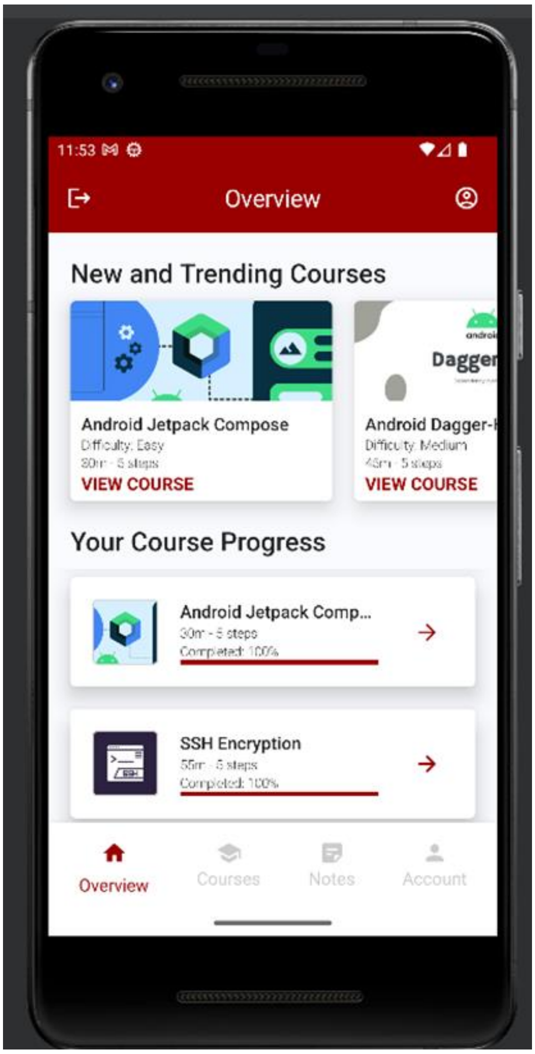

Fig 5.11 Course Overview Page

In the Course Overview page (Fig 5.11) of the e-learning Android project, users are presented with a list of courses they have enrolled in, along with their respective progress. The page features a visually appealing and organized layout, displaying the course cards or tiles in a grid or list format. Additionally, the progress of each enrolled course is prominently displayed, giving users an overview of their advancement in the course. This progress can be represented as progress bar, or any other visual indicator. The Course Overview page allows users to easily track their progress and access the enrolled courses, facilitating a seamless learning experience.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

In conclusion, the e-learning Android project has successfully developed a comprehensive and interactive e-learning application that offers a transformative educational experience. The project has addressed the evolving needs of learners by providing a user-friendly and feature-rich platform for accessing educational content anytime and anywhere.

Through meticulous planning and implementation, the project has created a diverse library of courses, encompassing various subjects and disciplines. Users can explore a wide range of topics and enroll in courses that cater to their specific interests and learning objectives. The application's intuitive interface makes it easy for users to navigate through the course catalog, access learning materials, and track their progress.

One of the project's key achievements is the integration of interactive learning tools and resources. The application incorporates multimedia elements, such as videos, quizzes, and interactive exercises, to enhance the learning experience and promote active engagement. Users can interact with the course content, participate in discussions, and collaborate with fellow learners, fostering a dynamic and immersive learning environment.

Furthermore, the project has implemented personalized features to cater to individual learning preferences. Users can customize their learning paths, set goals, and receive personalized recommendations based on their progress and interests. The application also offers comprehensive tracking and assessment tools, allowing users to monitor their learning achievements and receive feedback to improve their performance.

Overall, the e-learning Android project has successfully developed a robust and user-centric application that empowers learners to acquire knowledge and skills in a flexible and engaging manner. The project's dedication to accessibility, interactivity, and personalization has paved the way for a transformative e-learning experience, enabling individuals to pursue lifelong learning and personal growth.

FUTURE ENHANCEMENTS

1.Interactive Learning Activities: Introducing interactive learning activities such as quizzes, puzzles, or simulations to engage learners and enhance their understanding of the course material. These activities can provide hands-on practice and immediate feedback, making the learning experience more immersive and effective.

2.Social Learning Features: Adding social learning features that enable learners to connect and collaborate with peers, instructors, or experts in the field. This could include discussion forums, group projects, or live chat functionality, fostering a sense of community and facilitating knowledge sharing.

3.Personalized Learning Paths: Implementing personalized learning paths based on individual interests, goals, and learning styles. The e-learning platform could use algorithms and machine learning techniques to recommend courses, modules, or resources that align with each learner's preferences and needs.

4.Gamification Elements: Introducing gamification elements such as badges, leaderboards, or achievements to motivate learners and make the learning experience more enjoyable. These elements can encourage healthy competition, track progress, and provide a sense of accomplishment, enhancing learner engagement and retention.

5.Adaptive Learning Technologies: Incorporating adaptive learning technologies that dynamically adjust the course content and pace based on each learner's progress and performance. Adaptive learning algorithms can identify areas of strength and weakness, deliver personalized content, and provide targeted remediation to optimize learning outcomes.

6.Virtual Reality (VR) and Augmented Reality (AR) Integration: Integrating VR and AR technologies to create immersive learning experiences. This could involve virtual field trips, 3D simulations, or augmented reality overlays that allow learners to visualize complex concepts or interact with virtual objects, enhancing understanding and retention.

7.Data Analytics and Learning Analytics: Implementing robust data analytics and learning analytics capabilities to track learner progress, identify learning patterns, and generate actionable insights.

REFERENCES

- https://developer.android.com
- https://stackoverflow.com
- https://www.youtube.com
- https://www.udemy.com
- https://kotlinlang.org/docs